



(19) 대한민국특허청(KR)
(12) 등록특허공보(B1)

(45) 공고일자 2011년05월31일
(11) 등록번호 10-1037981
(24) 등록일자 2011년05월24일

(51) Int. Cl.

G06F 9/44 (2006.01) G06F 9/22 (2006.01)

G06F 9/06 (2006.01) G06F 9/30 (2006.01)

(21) 출원번호 10-2008-0137371

(22) 출원일자 2008년12월30일

심사청구일자 2008년12월30일

(65) 공개번호 10-2010-0078977

(43) 공개일자 2010년07월08일

(56) 선행기술조사문헌

KR1020070061112 A

JP2008288978 A

KR1020060070165 A

KR1020070096316 A

전체 청구항 수 : 총 4 항

(73) 특허권자

대구대학교 산학협력단

경북 경산시 진량읍 내리리 15 대구대학교 내

(72) 발명자

홍춘표

대구 달서구 상인동 보성은하아파트 106동 506호

김창훈

경남 통영시 평림동 851번지

박중태

대구광역시 북구 산격3동 경북대학교 공대 10호관 805호

(74) 대리인

이덕록

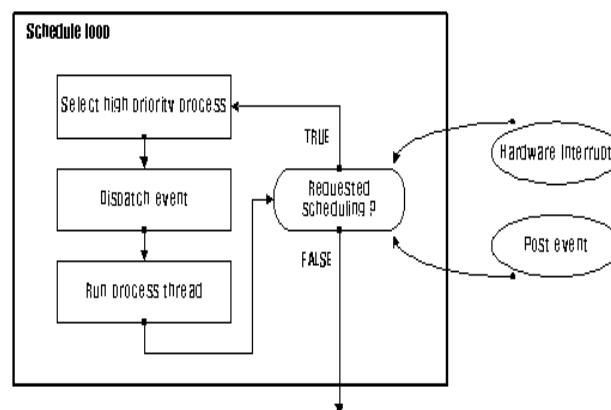
심사관 : 임영희

(54) 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법

(57) 요약

본 발명은 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법에 관한 것으로, 이벤트 트리븐 방식의 실행 모델에서 에이징(Aging) 기법을 적용한 우선순위 기반의 프로세스 스케줄링 방식을 사용하며 프로세스 사이의 상호작용은 동기적 또는 비동기적인 이벤트 전달로 이루어지고, 작업 수행의 기본 단위인 프로세스는 실행 중에 동적으로 로드(Load) 및 언로드(Unload)하는 것이 가능하며 프로세스 로더에 의한 동적 메모리 할당 및 프로그램 메모리 관리 기능을 제공함으로써 무선 네트워크를 이용한 원격 코드 업데이트의 수행을 용이하게 하는 원격 코드 업데이트가 용이하고, 원격 코드 업데이트를 수행할 때 새롭게 배포하고자 하는 어플리케이션 프로세스 이미지만을 전송하도록 하여 무선 네트워크를 통한 코드 업데이트에 소요되는 에너지와 시간을 절약할 수 있을 뿐만 아니라 안정적인 시스템을 유지하면서 센서 노드의 부족한 자원을 효율적으로 관리하고 시스템 유지 보수를 위한 효율적인 원격 코드 업데이트 기능을 지원함으로써 무선 센서 네트워크의 다양한 응용 플랫폼을 제공할 수 있는 각별한 장점이 있는 유용한 발명이다.

대 표 도 - 도6



이 발명을 지원한 국가연구개발사업

과제고유번호 C1090-0804-0004

부처명 지식경제부

연구관리전문기관

연구사업명 대학 IT연구센터 육성지원사업

연구과제명 u-헬스케어를 위한 고 신뢰 네트워크 및 의료정보 융합 기술 연구

기여율

주관기관 경북대학교

연구기간 2008년 06월 01일~2011년 12월 31일

특허청구의 범위

청구항 1

원격 코드 업데이트 매니저는 원격지의 노드들에게 새로운 프로세스 이미지 데이터를 전송하기 위해서 먼저 알림(ADVERTISEMENT) 메시지를 전송하는 단계와; 상기 알림 메시지를 받은 노드는 알림 메시지에 포함된 프로세스 이미지 데이터의 정보를 확인하여 프로세스 이미지 데이터 시스템에 이미 존재하는 프로세스 이미지 데이터이면 업데이트된 버전인지 확인하고 새로운 프로세스 이미지 데이터이면 해당 프로세스 이미지 데이터를 프로세스 이미지 데이터 시스템에 로드할 수 있는 충분한 공간이 있는지 확인하는 단계와; 다운로드 대기 상태에 있는 노드들이 다운로드(DOWNLOAD) 메시지와 함께 전송되는 프로세스 이미지 데이터를 수신하여 외부 플래시 메모리인 2차 저장 공간에 저장하는 단계와; 매니저가 마지막 캡슐(프로세스 이미지 데이터의 분할그룹)을 전송한 후 다운로드 완료(END_DOWNLOAD) 메시지를 전송하는 단계와; 원격지의 노드가 다운로드 완료 메시지를 수신하면 모든 캡슐(프로세스 이미지 데이터의 분할그룹)을 빠짐없이 수신하였는지 확인하고 손실되거나 수신하지 못한 캡슐이 존재하면 재요청을 통해서 프로세스 이미지 데이터 수신을 완료하는 단계와; 상기 프로세스 이미지 데이터가 프로세스 이미지 데이터 시스템으로 로드될 때 프로세스 이미지 데이터 로더는 먼저 프로세스 이미지 데이터에 포함된 정보를 바탕으로 프로그램과 데이터 메모리에 충분한 공간이 있는지 확인하는 단계와; 프로세스 이미지 데이터 로더가 프로세스 이미지 데이터를 로드할 프로그램 메모리 공간과 데이터 저장에 필요한 메모리 공간을 할당하고 할당된 데이터 메모리의 포인터를 프로세스 이미지 데이터에 전달하는 단계와; 프로세스 이미지 로드에 필요한 메모리 공간 할당이 완료되면 프로세스 이미지 데이터를 로드하는 단계 및; 프로세스 이미지 데이터 로드가 완료되면 EPRCU 커널은 초기화 이벤트와 함께 프로세스 이미지 데이터 스레드 함수를 호출하여 프로세스 이미지 데이터를 초기화하는 단계로 이루어진 것을 특징으로 하는 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법.

청구항 2

제1항에 있어서, 상기 원격 코드 업데이트 매니저는 원격지의 노드들에게 새로운 프로세스 이미지 데이터를 전송하기 위해서 먼저 알림(ADVERTISEMENT) 메시지를 전송하는 단계와 상기 알림 메시지를 받은 노드는 알림 메시지에 포함된 프로세스 이미지 데이터의 정보를 확인하여 프로세스 이미지 데이터 시스템에 이미 존재하는 프로세스 이미지 데이터이면 업데이트된 버전인지 확인하고 새로운 프로세스 이미지 데이터이면 해당 프로세스 이미지 데이터를 프로세스 이미지 데이터 시스템에 로드할 수 있는 충분한 공간이 있는지 확인하는 단계에서는 두 조건 중 한 가지라도 만족되면 (프로세스 이미지 데이터가 업데이트된 버전이거나 해당 프로세스 이미지 데이터를 프로세스 이미지 데이터 시스템에 로드할 수 있는 충분한 공간이 있으면) 상기 원격 코드 업데이트 매니저에게 프로세스 이미지 데이터 요청(REQUEST) 메시지를 전송하고 다운로드 대기 상태로 되고, 상기 원격 코드 업데이트 매니저는 알림 메시지를 일정 횟수 반복적으로 전송한 후 하나 이상의 노드로부터 프로세스 이미지 데이터 요청 메시지를 받으면 프로세스 이미지 데이터 전송을 시작하는 것을 특징으로 하는 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법.

청구항 3

삭제

청구항 4

제 1항에 있어서, 상기 프로세스 이미지 데이터가 프로세스 이미지 데이터 시스템으로 로드될 때 프로세스 이미지 데이터 로더는 먼저 프로세스 이미지 데이터에 포함된 정보를 바탕으로 프로그램과 데이터 메모리에 충분한 공간이 있는지 확인하는 단계에서는 두 메모리 중 어느 하나라도 공간이 부족하면 프로세스 이미지 데이터 로드는 중단되고, 프로세스 이미지 데이터 로더에 의해서 실행 중에 동적으로 프로세스 이미지 데이터 시스템으로 로드되는 프로세스 이미지 데이터는 프로세스 이미지 데이터 스레드 함수의 지역적 연속성을 유지하기 위해서 전역 변수나 정적 변수를 사용할 수 없으며, 프로세스 이미지 데이터에는 해당 프로세스 이미지 데이터가 수행에 필요한 데이터를 저장하기 위한 메모리 공간의 크기 정보가 기록되어 있는 것을 특징으로 하는 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법.

청구항 5

제 1항에 있어서, 상기 프로세스 이미지 데이터 로드에서 필요한 메모리 공간 할당이 완료되면 프로세스 이미지

데이터를 로드하는 단계에서는 프로세스 이미지 데이터 로더가 프로세스 이미지 데이터에 포함된 재배치 정보를 이용하여 외부 플래시 메모리에 저장된 프로세스 이미지 데이터를 상기 프로세스 이미지 데이터 로더가 프로세스 이미지 데이터를 로드할 프로그램 메모리 공간과 데이터 저장에 필요한 메모리 공간을 할당하고 할당된 데이터 메모리의 포인터를 프로세스 이미지 데이터에 전달하는 단계에서 할당한 프로그램 메모리에 링크하여 프로세스 이미지 데이터가 올바른 동작을 수행할 수 있도록 하는 것을 특징으로 하는 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법.

명세서

발명의 상세한 설명

기술 분야

[0001] 본 발명은 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법에 관한 것으로서, 더욱 상세하게는 이벤트 드리븐 방식의 실행 모델에서 에이징(Aging) 기법을 적용한 우선순위 기반의 프로세스 스케줄링 방식을 사용하며 프로세스 사이의 상호 작용은 동기적 또는 비동기적인 이벤트 전달로 이루어지고, 작업 수행의 기본 단위인 프로세스는 실행 중에 동적으로 로드(Load) 및 언로드(Unload) 하는 것이 가능하며 프로세스 로드에 의한 동적 메모리 할당 및 프로그램 메모리 관리 기능을 제공함으로써 무선 통신을 이용한 코드 업데이트 수행이 용이할 뿐만 아니라 다양한 센서 네트워크 응용에도 적합한 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법에 관한 것이다.

배경 기술

[0002] EPRCU(Easy to Perform Remote Code Update)는 전체 시스템을 도 1과 같이 하드웨어 의존 정도에 따라 4개(하드웨어, 디바이스 드라이버, 커널 서비스, 어플리케이션)의 논리적인 계층으로 구분한다. 커널 서비스 계층이 어플리케이션 계층에 하드웨어 추상화를 제공하고 있으며 디바이스 드라이버와 어플리케이션이 직접적으로 통신하는 것은 허용하지 않는다.

[0003] EPRCU 시스템은 커널 라이브러리, 디바이스 드라이버, 서비스 프로세스, 그리고 로드된 프로세스들로 구성된다. 로드된 프로세스는 어플리케이션 프로그램 또는 커널 서비스일 수 있다. 어플리케이션 프로그램은 하나 이상의 프로세스로 구성될 수 있으며 모든 프로세스는 실행 중에 동적으로 교체되거나 삭제될 수 있다. 프로세스들 사이의 통신은 동기적인 이벤트를 포스팅(Posting)하는 방법을 사용하며 항상 커널을 통해서만 이루어진다. 이벤트는 파라미터와 함께 프로세스에 포스팅되고 프로세스 스레드에 의해서 처리된다. 프로세스의 모든 상태는 15 바이트로 구성된 프로세스 제어 블록(Process Control Block)에 저장되고 커널은 프로세스 제어 블록에 대한 포인터만을 유지하고 이를 통해서 프로세스를 관리한다. 단일 스택을 사용하는 EPRCU는 모든 프로세스들이 같은 주소공간을 공유하며 다른 보호 도메인에서 동작하지 않는다.

[0004] 실행중인 EPRCU 시스템은 도 2와 같이 커널 코어와 로드된 프로세스영역으로 분할되어 있다. 커널 코어 영역은 디바이스 드라이버, 커널 라이브러리, 커널 서비스 프로세스 그리고 프로세스 로더로 구성되며 하드웨어 구성에 따라 사용되는 코드만을 포함하여 컴파일 된 단일 이미지 형태로 전송된다. 일반적으로 커널 코어는 한번 전송되면 수정이 불가능하며 이를 가능하도록 하기 위해서는 특수한 부트로더 모듈을 필요로 한다.

[0005] 프로세스는 프로세스 로더에 의해서 시스템으로 로드된다. 프로세스 로더는 무선 및 시리얼 통신 그리고 노드에 직접 연결된 외부 플래시 메모리로부터 프로세스 이미지를 얻을 수 있다. 그러나 일반적으로 시스템의 안정성을 고려하여 무선 및 시리얼 통신으로부터 얻은 프로세스 이미지를 코드 메모리에 프로그램하기 전에 외부 플래시 메모리에 먼저 저장하고 외부 플래시 메모리로부터 다시 프로세스 이미지를 얻는 방식을 사용한다.

[0006] EPRCU의 커널은 대기 중인 프로세스에 이벤트를 전달하는 이벤트 스케줄러와 통신 스택을 포함한 커널 서비스 프로세스로 구성된다. 모든 프로세스의 실행은 이벤트 발생에 의해서 동작한다. 커널은 한번 스케줄 된 프로세스를 선점하지 않기 때문에 프로세스 스레드는 Run-to-Completion 하다고 할 수 있다. 그러나 프로세스는 하드웨어 인터럽트에 의해서 선점당할 수 있다.

[0007] 커널은 동기와 비동기적인 두 가지 이벤트 유형을 제공한다. 비동기적인 이벤트들은 커널에 의해 이벤트 큐에 삽입되고 다음 스케줄링을 통해서 타겟 프로세스에 전달되는 형태이다. 동기적인 이벤트들은 비동기 이벤트들과 비슷하지만 타겟 프로세스에 즉시 전달되고 실행의 제어 권을 스케줄 된 프로세스가 이벤트 처리를 완료한 후에

반환되기 때문에 프로세스들 사이의 통신을 가능하게 한다.

[0008] EPRCU 커널에서는 모든 실행 프로세스들이 하나의 동일한 스택을 사용함으로써 부족한 메모리 자원을 효율적으로 공유한다. 또한 비동기 이벤트의 사용은 각 이벤트를 처리하는 프로세스 스레드들 사이에서 스택을 리와인드(rewind) 함으로써 스택 공간의 요구사항과 오버플로우 발생을 줄인다. 다중 프로세스 시스템에서 다중 스택을 사용하는 경우와 비교해서 이벤트 드리븐 시스템에서 단일 스택을 사용하였을 때에는 여러 가지 이점을 얻을 수 있다.

[0009] 이벤트 드리븐 시스템에서 모든 실행 프로세스들이 단일 스택을 공유하여 사용하면 다중 프로세스 시스템에서 다중 스택을 사용하는 것보다 센서 노드상의 부족한 메모리 자원을 효율적으로 사용할 수 있다. 도 3은 3개의 프로세스가 존재하는 다중 프로세스 시스템과 이벤트 드리븐 시스템에서 각 프로세스가 자신의 고유한 스택을 가지는 경우(도 3a)와 하나의 동일한 스택을 공유하는 경우(도 3b)를 나타낸 것이다. 도 3a와 도 3b 두 경우 모두 각 스레드가 요구하는 최소 스택 메모리량과 실행 중에 사용하게 되는 최대 스택 메모리량은 동일하다. 도 3a의 경우 프로세스는 스택 메모리를 이용하여 작업을 수행하고 작업을 완료하면 사용했던 메모리를 반납한 후 다른 프로세스가 작업을 수행하기 때문에 특정 시점에서 스택 메모리의 전체 사용량은 현재 동작중인 프로세스의 스택 메모리 사용량과 동일하다. 그러나 도 3b의 경우 각 프로세스들은 자신만의 고유한 스택 공간을 가지며 서로 독립적으로 동작하기 때문에 특정 시점에서 스택 메모리의 전체 사용량은 각 프로세스가 사용 중인 메모리를 모두 합한 것과 같다. 즉 기본적으로 도 3a가 도 3b 보다 더 많은 메모리 자원을 필요로 한다. 따라서 극도로 제한된 메모리 공간을 가지는 센서 노드 환경에서는 도 3a 보다 도 3b가 더 적합하다.

[0010] EPRCU의 프로세스는 프로세스 제어 블록과 스레드 함수로 구성된다. 프로세스 제어 블록에는 실행중인 프로세스에 대한 여러 가지 정보가 저장된다. 도 4는 EPRCU의 프로세스 제어 블록과 제어 블록을 구성하는 각 필드들을 나타낸 것이다.

[0011] 제어 블록을 구성하는 각 필드는 초기에 컴파일 시간에 그 값이 결정되고 실행 중에 커널에 의해서 변경될 수 있다. 각 필드가 가지는 의미와 용도를 하기 표 1에 나타내었다.

표 1

[0012] 프로세스 제어 블록을 구성하는 각 필드의 의미와 용도

Field name	Description
ID	■프로세서 ID ■실행중인 커널에서 각 프로세스를 구분하는 고유한 식별자
name	■문자열로 이루어진 프로세스 이름 ■ID와 함께 프로세스를 구분하는 식별자 ■디버깅 모드에서 동작할 때 실행의 흐름을 파악하기 위한 디버깅 용도로도 사용됨.
Thread	■프로세스 스레드 함수의 포인터
Original priority	■컴파일 시간에 결정되는 프로세스의 기본 우선순위
State	■프로세스의 상태 ■실행 중에 커널에 의해서 그 값이 변경됨.
Aging priority	■프로세스의 에이징 우선순위 ■프로세스가 READY로 변경될 때 기본 우선순위로 결정. ■에이징 카운터 값에 따라 한번에 1씩 증가 ■스케줄러가 가장 높은 우선순위를 가지는 프로세스를 선정할 때 각 프로세스의 우선순위를 비교하는 기준이 됨
Aging count	■프로세스의 에이징 카운트 ■에이징 타이머에 의해서 한번에 1씩 증가 ■카운트 값이 일정 이상이 되면 에이징 우선순위 증가
Event list	■프로세스가 받은 이벤트 리스트 포인터 ■프로세스는 받은 이벤트를 리스트 형태로 관리하기 때문에 하나 이상의 이벤트를 받는 것이 가능함
Wait point	■스레드 함수의 지역적 연속성을 유지하기 위한 포인터
Next	■프로세스 리스트를 위한 다음 제어 블록에 대한 포인터

[0018] 프로세스 스레드 함수의 지역적 연속성 구현을 위한 매크로 함수

```

#define LWT_INIT()          WP_INIT((process_wp)->wp)
#define LWT_START()      {
                                uint8_t wait_flag = 0; \
                                WP_RESUME((process_wp)->wp)
#define LWT_END()          WP_END((process_wp)->wp); \
                                wait_flag = 1; \
                                LWT_INIT(); \
                                return LWT_STATE_ENDED; \
                                }
#define LWT_EXIT()
#define LWT_WAIT()
                                do {
                                    wait_flag = 1; \
                                    WP_SET((process_wp)->wp); \
                                    if(wait_flag == 1) return LWT_STATE_WAITING; \
                                } while(0)
#define LWT_WAIT_CONDITION(condition) \
                                do {
                                    WP_SET((process_wp)->wp); \
                                    if(!condition) return LWT_STATE_WAITING; \
                                } while(0)
#define LWT_WAIT_EVENT(event_number) \
                                do {
                                    wait_flag = 1; \
                                    WP_SET((process_wp)->wp); \
                                    if((wait_flag == 1) || (event_number != event_type)) \
                                    { \
                                        return LWT_STATE_WAITING; \
                                    } \
                                } while(0)

```

[0019] 실행중인 EPRCU 시스템에는 정적으로 링크된 프로세스와 동적으로 링크된 프로세스가 함께 존재한다. 정적으로 링크된 프로세스는 커널과 함께 컴파일되고 커널 이미지에 함께 포함되어 전송되기 때문에 프로세스 스레드 함수에서 추가적인 작업 없이 커널 API를 사용할 수 있고 커널에서도 일반적인 방법으로 프로세스 스레드 함수를 실행시킬 수 있다.

[0020] 정적으로 링크된 프로세스와는 달리 동적으로 링크되는 프로세스는 커널과는 별도로 컴파일되고 시리얼 또는 무선 통신을 이용하여 실행중인 노드에 배포된다. 따라서 정적으로 링크된 프로세스와 같이 일반적인 방법으로는 커널 API를 사용하는 것이 불가능하며 실행 중에 노드로 다운로드 된 프로세스가 프로그램 메모리의 어느 위치에 적재될지 알 수 없기 때문에 커널이 프로세스 스레드 함수를 올바르게 호출하기 위해서는 추가적인 작업이 필요하다. EPRCU 커널은 도 5와 같이 커널 API 점프 테이블을 이용하여 시스템으로 새롭게 로드된 프로세스에서도 커널 API를 호출할 수 있도록 한다. 또한 커널은 필요에 따라 점프 테이블에 새로운 항목을 추가하거나 기존에 존재하는 항목을 삭제하는 것이 가능하다.

[0021] 동적으로 링크되는 프로세스는 실행 중에 재배치될 함수의 코드와 재배치 정보를 포함한 이미지로 구성된다. 프로세스 로더는 재배치 정보를 이용하여 시스템으로 프로세스를 로드하기 위해 필요한 메모리 공간을 할당하고 함수들을 재배치한다.

[0022] EPRCU 시스템에서 새로운 프로세스가 원격 코드 업데이트 매니저로부터 실행중인 원격지의 노드로 전송되는 과정은 다음과 같다. 커널은 필요에 따라 점프 테이블에 새로운 항목을 추가하거나 기존에 존재하는 항목을 삭제하는 것이 가능하다.

[0023] 동적으로 링크되는 프로세스는 실행 중에 재배치될 함수의 코드와 재배치 정보를 포함한 이미지로 구성된다. 프

로세스 로더는 재배치 정보를 이용하여 시스템으로 프로세스를 로드하기 위해 필요한 메모리 공간을 할당하고 함수들을 재배치한다.

발명의 내용

해결 하고자하는 과제

- [0024] 본 발명은 상기한 실정을 감안하여 발명한 것으로서, 그 목적은 무선 네트워크를 이용한 원격 코드 업데이트의 수행을 용이하게 하는 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법을 제공함에 있다.
- [0025] 본 발명의 다른 목적은 원격 코드 업데이트를 수행할 때 새롭게 배포하고자 하는 어플리케이션 프로세스 이미지만을 전송하도록 하여 무선 네트워크를 통한 코드 업데이트에 소요되는 에너지와 시간을 절약할 수 있는 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법을 제공하는 데 있다.
- [0026] 본 발명의 또 다른 목적은 안정적인 시스템을 유지하면서 센서 노드의 부족한 자원을 효율적으로 관리하고 시스템 유지 보수를 위한 효율적인 원격 코드 업데이트 기능을 지원함으로써 무선 센서 네트워크의 다양한 응용 플랫폼을 제공할 수 있는 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법을 제공하는 데 있다.

과제 해결수단

- [0027] 상기한 목적을 달성하기 위한 본 발명 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법은 원격 코드 업데이트 매니저는 원격지의 노드들에게 새로운 프로세스 이미지 데이터를 전송하기 위해서 먼저 알림(ADVERTISEMENT) 메시지를 전송하는 단계와; 상기 알림 메시지를 받은 노드는 알림 메시지에 포함된 프로세스 이미지 데이터의 정보를 확인하여 프로세스 이미지 데이터 시스템에 이미 존재하는 프로세스 이미지 데이터이면 업데이트된 버전인지 확인하고 새로운 프로세스 이미지 데이터이면 해당 프로세스 이미지 데이터를 프로세스 이미지 데이터 시스템에 로드할 수 있는 충분한 공간이 있는지 확인하는 단계와; 다운로드 대기 상태에 있는 노드들이 다운로드(DOWNLOAD) 메시지와 함께 전송되는 프로세스 이미지 데이터를 수신하여 외부 플래시 메모리인 2차 저장 공간에 저장하는 단계와; 매니저가 마지막 캡슐(프로세스 이미지 데이터의 분할그룹)을 전송한 후 다운로드 완료(END_DOWNLOAD) 메시지를 전송하는 단계와; 원격지의 노드가 다운로드 완료 메시지를 수신하면 모든 캡슐(프로세스 이미지 데이터의 분할그룹)을 빠짐없이 수신하였는지 확인하고 손실되거나 수신하지 못한 캡슐이 존재하면 재요청을 통해서 프로세스 이미지 데이터 수신을 완료하는 단계와; 상기 프로세스 이미지 데이터가 프로세스 이미지 데이터 시스템으로 로드될 때 프로세스 이미지 데이터 로더는 먼저 프로세스 이미지 데이터에 포함된 정보를 바탕으로 프로그램과 데이터 메모리에 충분한 공간이 있는지 확인하는 단계와; 프로세스 이미지 데이터 로더가 프로세스 이미지 데이터를 로드할 프로그램 메모리 공간과 데이터 저장에 필요한 메모리 공간을 할당하고 할당된 데이터 메모리의 포인터를 프로세스 이미지 데이터에 전달하는 단계와; 프로세스 이미지로드에 필요한 메모리 공간 할당이 완료되면 프로세스 이미지 데이터를 로드하는 단계 및; 프로세스 이미지 데이터 로드가 완료되면 EPRCU 커널은 초기화 이벤트와 함께 프로세스 이미지 데이터 스레드 함수를 호출하여 프로세스 이미지 데이터를 초기화하는 단계로 이루어진 것을 특징으로 한다.

효 과

- [0028] 본 발명은 무선 네트워크를 이용한 원격 코드 업데이트의 수행을 용이하게 하는 원격 코드 업데이트가 용이하고, 원격 코드 업데이트를 수행할 때 새롭게 배포하고자 하는 어플리케이션 프로세스 이미지만을 전송하도록 하여 무선 네트워크를 통한 코드 업데이트에 소요되는 에너지와 시간을 절약할 수 있을 뿐만 아니라 안정적인 시스템을 유지하면서 센서 노드의 부족한 자원을 효율적으로 관리하고 시스템 유지 보수를 위한 효율적인 원격 코드 업데이트 기능을 지원함으로써 무선 센서 네트워크의 다양한 응용 플랫폼을 제공할 수 있는 각별한 장점이 있다.

발명의 실시를 위한 구체적인 내용

- [0029] 이하, 첨부 도면을 참조하여 본 발명 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법의 바람직한 실시예를 상세하게 설명한다.
- [0030] 도 3a는 본 발명에 따른 다중 프로세스 시스템에서 다중 스택의 메모리 요구사항을 나타낸 도면, 도 3b는 본 발명에 따른 다중 프로세스 시스템에서 단일 스택의 메모리 요구사항을 나타낸 도면, 도 4는 본 발명에 따른 프로

세스 제어블록의 구성도, 도 5는 본 발명에 따른 커널 API 점프 테이블의 사용 예시도, 도 6은 본 발명에 따른 EPRCU 프로세스 스케줄러의 구조도, 도 7은 본 발명에 따른 에이징 타이머를 이용한 프로세싱 에이징을 나타낸 도면, 도 8은 본 발명에 따른 UTRC SN100 센서 노드와 AVR JTAG ICE mk-II의 구성도로서, 본 발명 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법은 원격 코드 업데이트 매니저는 원격지의 노드들에게 새로운 프로세스 이미지 데이터를 전송하기 위해서 먼저 알림(ADVERTISEMENT) 메시지를 전송하는 단계와; 상기 알림 메시지를 받은 노드는 알림 메시지에 포함된 프로세스 이미지 데이터의 정보를 확인하여 프로세스 이미지 데이터 시스템에 이미 존재하는 프로세스 이미지 데이터이면 업데이트된 버전인지 확인하고 새로운 프로세스 이미지 데이터이면 해당 프로세스 이미지 데이터를 프로세스 이미지 데이터 시스템에 로드할 수 있는 충분한 공간이 있는지 확인하는 단계와; 다운로드 대기 상태에 있는 노드들이 다운로드(DOWNLOAD) 메시지와 함께 전송되는 프로세스 이미지 데이터를 수신하여 외부 플래시 메모리인 2차 저장 공간에 저장하는 단계와; 매니저가 마지막 캡슐(프로세스 이미지 데이터의 분할그룹)을 전송한 후 다운로드 완료(END_DOWNLOAD) 메시지를 전송하는 단계와; 원격지의 노드가 다운로드 완료 메시지를 수신하면 모든 캡슐(프로세스 이미지 데이터의 분할그룹)을 빠짐없이 수신하였는지 확인하고 손실되거나 수신하지 못한 캡슐이 존재하면 재요청을 통해서 프로세스 이미지 데이터 수신을 완료하는 단계와; 상기 프로세스 이미지 데이터가 프로세스 이미지 데이터 시스템으로 로드될 때 프로세스 이미지 데이터 로더는 먼저 프로세스 이미지 데이터에 포함된 정보를 바탕으로 프로그램과 데이터 메모리에 충분한 공간이 있는지 확인하는 단계와; 프로세스 이미지 데이터 로더가 프로세스 이미지 데이터를 로드할 프로그램 메모리 공간과 데이터 저장에 필요한 메모리 공간을 할당하고 할당된 데이터 메모리의 포인터를 프로세스 이미지 데이터에 전달하는 단계와; 프로세스 이미지 로드에 필요한 메모리 공간 할당이 완료되면 프로세스 이미지 데이터를 로드하는 단계 및; 프로세스 이미지 데이터 로드가 완료되면 EPRCU 커널은 초기화 이벤트와 함께 프로세스 이미지 데이터 스레드 함수를 호출하여 프로세스 이미지 데이터를 초기화하는 단계로 이루어져 있다.

[0031] 상기 원격 코드 업데이트 매니저는 원격지의 노드들에게 새로운 프로세스 이미지 데이터를 전송하기 위해서 먼저 알림(ADVERTISEMENT) 메시지를 전송하는 단계와 상기 알림 메시지를 받은 노드는 알림 메시지에 포함된 프로세스 이미지 데이터의 정보를 확인하여 프로세스 이미지 데이터 시스템에 이미 존재하는 프로세스 이미지 데이터이면 업데이트된 버전인지 확인하고 새로운 프로세스 이미지 데이터이면 해당 프로세스 이미지 데이터를 프로세스 이미지 데이터 시스템에 로드할 수 있는 충분한 공간이 있는지 확인하는 단계에서는 두 조건 중 한 가지라도 만족되면 (프로세스 이미지 데이터가 업데이트된 버전이거나 해당 프로세스 이미지 데이터를 프로세스 이미지 데이터 시스템에 로드할 수 있는 충분한 공간이 있으면) 상기 원격 코드 업데이트 매니저에게 프로세스 이미지 데이터 요청(REQUEST) 메시지를 전송하고 다운로드 대기 상태로 되고, 상기 원격 코드 업데이트 매니저는 알림 메시지를 일정 횟수 반복적으로 전송한 후 하나 이상의 노드로부터 프로세스 이미지 데이터 요청 메시지를 받으면 프로세스 이미지 데이터 전송을 시작한다.

[0032] 상기 다운로드 대기 상태에 있는 노드들이 다운로드(DOWNLOAD) 메시지와 함께 전송되는 프로세스 이미지 데이터를 수신하여 외부 플래시 메모리인 2차 저장 공간에 저장한다.

[0033] 상기 프로세스 이미지 데이터가 프로세스 이미지 데이터 시스템으로 로드될 때 프로세스 이미지 데이터 로더는 먼저 프로세스 이미지 데이터에 포함된 정보를 바탕으로 프로그램과 데이터 메모리에 충분한 공간이 있는지 확인하는 단계에서는 두 메모리 중 어느 하나라도 공간이 부족하면 프로세스 이미지 데이터 로드는 중단되고, 프로세스 이미지 데이터 로더에 의해서 실행 중에 동적으로 프로세스 이미지 데이터 시스템으로 로드되는 프로세스 이미지 데이터는 프로세스 이미지 데이터 스레드 함수의 지역적 연속성을 유지하기 위해서 전역 변수나 정적 변수를 사용할 수 없으며, 프로세스 이미지 데이터에는 해당 프로세스 이미지 데이터가 수행에 필요한 데이터를 저장하기 위한 메모리 공간의 크기 정보가 기록되어 있다.

[0034] 그리고 상기 프로세스 이미지 데이터 로드에 필요한 메모리 공간 할당이 완료되면 프로세스 이미지 데이터를 로드하는 단계에서는 프로세스 이미지 데이터 로더가 프로세스 이미지 데이터에 포함된 재배치 정보를 이용하여 외부 플래시 메모리에 저장된 프로세스 이미지 데이터를 상기 프로세스 이미지 데이터 로더가 프로세스 이미지 데이터를 로드할 프로그램 메모리 공간과 데이터 저장에 필요한 메모리 공간을 할당하고 할당된 데이터 메모리의 포인터를 프로세스 이미지 데이터에 전달하는 단계에서 할당한 프로그램 메모리에 링크하여 프로세스 이미지 데이터가 올바른 동작을 수행할 수 있도록 한다.

[0035] 다음에는 상기한 바와 같이 이루어지는 본 발명 원격 코드 업데이트가 용이한 무선 센서 노드용 운영체제 구현방법의 작용을 상세하게 설명한다.

[0036] EPRCU의 이벤트는 프로세스 이미지 데이터와 프로세스 이미지 데이터 또는 프로세스 이미지 데이터와 커널 사이

의 상호작용을 위한 수단으로 사용된다. 이벤트의 생성 및 전달은 오직 프로세스 이미지 데이터 레벨에서만 가능하며 특수한 경우(UART 데이터 수신, RADIO 패킷 수신)를 제외하고 인터럽트 레벨에서 이벤트와 관련된 작업 수행을 제한한다. 또한 이벤트는 커널 또는 프로세스 이미지 데이터 레벨에서 작업의 시작이나 완료 등과 같이 특정한 사건의 발생을 특정 프로세스 이미지 데이터 또는 커널에 알리기 위한 용도로 사용된다는 점에서 하드웨어 인터럽트와는 구분된다.

- [0037] EPRCU는 동기와 비동기적인 두 가지 이벤트 유형을 제공한다. 비동기적인 이벤트들은 커널에 의해 이벤트 큐에 삽입되고 다음 스케줄링을 통해서 타겟 프로세스 이미지 데이터에 전달되는 형태이다. 동기적인 이벤트들은 비동기 이벤트들과 비슷하지만 타겟 프로세스 이미지 데이터에 즉시 전달되고 실행의 제어권을 스케줄 된 프로세스 이미지 데이터가 이벤트 처리를 완료한 후에 반환되기 때문에 프로세스 이미지 데이터들 사이의 통신을 가능하게 한다. 표 4는 EPRCU 커널이 제공하는 동기와 비동기적인 이벤트를 프로세스 이미지 데이터에 포스팅하는 커널 API를 나타낸 것이다.

표 4

- [0038] 동기와 비동기 이벤트 전달 API

■void process_post_event_sync(process_t *p, process_event_t e, process_event_parameter_t param); ■동기적인 이벤트 전달
■void process_post_event_async(process_t *p, process_event_t e, process_event_parameter_t param); ■비동기적인 이벤트 전달

- [0039] 비동기적인 이벤트 전달 및 처리를 위해서 커널은 이벤트 타입, 파라미터, 다음 이벤트 객체를 가리키는 포인터 이렇게 3개의 필드를 가지는 이벤트 객체로 구성된 큐를 가지고 있다. 프로세스 이미지 데이터가 작업을 수행하는 도중에 이벤트가 발생하면 커널의 이벤트 큐에 해당 이벤트와 관련된 정보(이벤트 타입, 파라미터)를 저장하고 발생 이벤트를 이벤트의 타겟으로 지정된 프로세스 이미지 데이터의 이벤트 리스트에 삽입한다. 삽입된 이벤트는 이벤트 스케줄러에 의해서 해당 프로세스 이미지 데이터가 실행을 재개할 때 프로세스 이미지 데이터의 이벤트 리스트에 대기 중인 순서대로 처리된다.

- [0040] 동기적인 이벤트 전달 및 처리 방법은 비동기적인 이벤트 전달 및 처리와는 달리 별도의 이벤트 저장 공간을 필요로 하지 않는다. 이벤트가 발생하면 수행 중이던 프로세스 이미지 데이터의 작업을 잠시 중단하고 이벤트의 타겟으로 지정된 프로세스 이미지 데이터가 즉시 수행을 시작하도록 하여 발생된 이벤트가 즉각적으로 처리되도록 한다. 이벤트 처리가 완료된 후에는 원래 수행중이었던 프로세스 이미지 데이터의 작업을 재개한다. 이 방법은 이벤트의 발생시점으로부터 처리시작까지의 지연을 최소화하여 실시간성이 요구되는 이벤트 처리에 적합하다.

- [0041] EPRCU에서 모든 프로세스 이미지 데이터에 대한 이벤트 스케줄링은 단일 레벨에서 완료되고 서로 선점할 수 없다. 프로세스 이미지 데이터 이벤트는 오직 하드웨어 인터럽트에 의해서만 선점될 수 있다.

- [0042] 커널은 명확한 스케줄링 포인트를 위한 플래그를 제공한다. 스케줄링 플래그는 하드웨어 인터럽트의 발생이나 실행중인 프로세스 이미지 데이터의 이벤트 포스팅으로 인해 설정될 수 있다. 모든 프로세스 이미지 데이터는 우선순위를 가지며 커널은 이벤트가 발생하거나 스케줄링 플래그가 설정되면 프로세스 이미지 데이터 리스트에 존재하는 프로세스 이미지 데이터들의 우선순위를 비교하여 가장 높은 우선순위를 가지는 프로세스 이미지 데이터를 선택한다. 스케줄러는 선택된 프로세스 이미지 데이터의 이벤트 리스트에 처리되지 않은 이벤트가 존재하면 프로세스 이미지 데이터 스레드 함수를 실행할 때 이벤트를 함께 전달한다. 해당 프로세스 이미지 데이터에 대기 중인 이벤트가 존재하거나 프로세스 이미지 데이터 리스트에 대기 중인 프로세스 이미지 데이터가 존재하면 스케줄링 플래그를 설정하여 스케줄링 루프를 빠져나가지 않도록 한다. 도 6은 EPRCU 프로세스 스케줄러의 구조를 나타낸 것이다.

- [0043] 에이징 기법은 SJF(Shortest Job First) 또는 우선순위 기반의 스케줄링 기법을 사용하는 시스템에서 특정 프로세스 이미지 데이터의 우선순위가 낮아 스케줄러에 의해 선택되지 못하고 무한정 기다리게 되는 프로세스 이미지 데이터 기아(Starvation)를 방지하기 위하여, 한번 양보하거나 기다린 시간에 비례하여 일정 시간이 지나면 우선순위를 한 단계씩 높여 가까운 시간 안에 작업의 수행을 시작할 수 있도록 하는 것이다.

[0044] EPRCU는 이벤트 드리븐 방식의 실행 모델에서 우선순위 기반의 프로세스 이미지 데이터 스케줄링 기법을 사용하지만 에이징 기법을 적용하여 우선순위가 낮은 특정 프로세스 이미지 데이터가 무한 연기 또는 기아 상태에 빠지는 것을 방지한다. 이를 위해 각 프로세스 이미지 데이터는 제어 블록 내에 에이징된 우선순위와 에이징 카운트를 저장하는 변수를 가진다. 프로세스 이미지 데이터가 READY 상태로 전환될 때 에이징 우선순위는 해당 프로세스 이미지 데이터의 기본 우선순위 값으로 설정되고 에이징 카운터는 0으로 초기화된다. 도 7과 같이 에이징 타이머는 스케줄링 루프로 들어가기 전에 활성 상태가 되고 커널이 지정한 시간이 만료되면 프로세스 이미지 데이터 리스트에서 READY 상태로 존재하는 모든 프로세스 이미지 데이터들의 에이징 카운트를 1씩 증가시키고 그 카운트 값에 따라 우선순위를 조절한다(현재 구현된 시스템에서는 2 이상일 경우). 커널은 프로세스 이미지 데이터 리스트에 READY 상태의 프로세스 이미지 데이터가 더 이상 존재하지 않으면 스케줄링 루프를 빠져나오게 되며 이때 에이징 타이머를 비활성 상태로 변경한다.

[0045] 센서 네트워크에서 시스템에 더 이상 수행할 작업이 없거나 네트워크가 유흟상태일 때 센서 노드의 에너지 소비를 줄이기 위해서 CPU를 저전력 모드로 전환하는 것이 필요하다. 노드의 에너지 소비량은 어플리케이션과 네트워크 프로토콜에 따라 다를 수 있다.

[0046] EPRCU는 더 이상 시스템에 이벤트 스케줄러가 처리해야 할 이벤트가 존재하지 않으면 스케줄링 루프를 종료하고 저전력 모드로 전환한다. 저전력 모드 상태의 CPU는 인터럽트에 의해서 깨어날 수 있으며 시스템은 다시 스케줄링 루프를 시작하여 발생한 이벤트를 처리함으로써 센서 노드상의 제한된 에너지 자원을 효율적으로 사용할 수 있다.

[0047] 실시예

[0048] 본 발명의 방법으로 새로운 무선 센서 노드용 운영체제인 EPRCU를 구현하고 UTRC(Ubiquitous Technology Research Center)의 SN100 센서 보드와 AVR JTAG ICE mk II를 이용하여 테스트 및 성능 평가를 수행하였다. 도 8과 표 5는 테스트에 사용된 SN100 센서 노드의 하드웨어 구성을 나타낸 것이다.

표 5

[0049] UTEC SN100 센서 노드의 하드웨어 구성

Mote Hardware Platform		UTRC SN100
MCU	Chip	ATmega128 }
	Type	8MHz, 8bit
	Program Memory(kB)	128
	SRAM(kB)	4
RF Transceiver (Radio)	Chip	CC2420
	Radio Frequency(MHz)	2400
	Max. Data Rate(kbits/sec)	250
	Antenna Connector	MMCX
Flash Data Logger Memory	Chip	AT45DB014B
	Connection Type	SPI
	Size(kB)	512
Default power source	Type	AA, 2x
	Typical capacity(mA-hr)	2000
	3.3V booster	N/A

[0050] 표 6에는 프로세스 사이의 상호작용과 커널 및 프로세스 이미지 데이터의 수행을 위한 주요 API에 소요되는 CPU 사이클을 나타내었으며, 표 7에는 프로세스 이미지 데이터 로더에 의해서 수행되는 메모리 관리 API에 소요되는 CPU 사이클을 나타내었다.

표 6

[0051] EPRCU 주요 커널 API에 소요되는 CPU 사이클

Communication method	Clock cycles
Post synchronous event	54
Post asynchronous event	186
Dispatch event from scheduler	137
Call using kernel jump table	12
Direct function call	4

표 7

[0052] EPRCU 주요 커널 API에 소요되는 CPU 사이클

Memory management	Clock cycles
Allocate data memory	64
Deallocate data memory	92
Allocate code memory	88
Deallocate code memory	109
Change memory owner	48
Check memory crash	1124

[0053] 또한 기존에 개발된 운영체제 중 원격 코드 업데이트 기능을 포함하는 시스템과 제안된 시스템의 메모리(ROM, RAM) 사용량을 측정하여 그 결과를 표 8에 나타내었다. 비교 결과 EPRCU는 기존에 개발된 시스템에 비해서 약 2 ~ 3 배 적은 프로그램 메모리와 1K 미만의 데이터 메모리만으로도 원격 코드 업데이트 기능을 수행할 수 있다는 것을 확인하였다.

표 8

[0054] 원격 코드 업데이트 기능을 포함하는 운영체제의 메모리 사용량 비교

Platform	Program memory(ROM)	Data memory(RAM)
SOS	20464 B	1163 B
TinyOS with deluge	21132 B	597 B
Bombilla virtual Machine	39746 B	3196 B
EPRCU Core	11566 B	857 B

[0055] 이상 설명한 바와 같이 무선 센서 네트워크를 위한 운영체제는 자원 제약적인 센서 노드 환경에서 효율적으로 자원을 관리하고 센서 네트워크상에서의 다양한 응용 환경을 제공해야 할 뿐만 아니라 이미 전송된 네트워크에서 응용 프로그램의 오류가 발생하여 이를 수정하거나 성능 개선을 위해서 무선 네트워크를 이용한 안전한 코드 업데이트 기능도 제공해야 한다. 이를 위해서 본 발명에서는 원격 코드 업데이트가 용이한 새로운 센서 노드용 운영체제, EPRCU를 제안하였다.

[0056] EPRCU는 이벤트 드리븐 방식의 실행 모델에서 우선순위 기반의 프로세스 스케줄링 방식을 사용하였다. GCC C 컴파일러의 'labels-as-values' 확장 문법을 이용하여 프로세스 이미지 데이터 스레드 함수의 지역적 연속성을 유지하였고 이를 이용하여 프로세스 이미지 데이터 스스로가 CPU를 양보하고 다음 스케줄링을 통해서 긴 작업을 계속 이어가도록 함으로써 단일 공유 스택을 사용하는 이벤트 드리븐 시스템의 장점을 유지하고 단점을 보완하였다. 또한 에이징 기법을 적용하여 우선순위 기반의 스케줄링 기법을 사용하는 시스템에서 우선순위가 낮은 특정 프로세스 이미지 데이터가 스케줄러에 의해 선택되지 못하고 무한 연기 또는 기아 상태에 빠지는 것을 방지하였다.

[0057] 프로세스 이미지 데이터 로더는 작업 수행의 기본 단위인 프로세스 이미지 데이터를 실행 중 동적으로 시스템에 로드 및 언로드하는 것이 가능하며 새롭게 시스템으로 로드 또는 언로드되는 프로세스 이미지 데이터를 위한 데이터 메모리와 프로그램 메모리 관리 기능을 제공한다. 이는 무선 네트워크를 이용한 원격 코드 업데이트의 수행을 용이하도록 할 뿐만 아니라 원격 코드 업데이트를 수행할 때 새롭게 배포하고자 하는 어플리케이션 프로세스 이미지 데이터만을 전송하도록 하여 무선 네트워크를 통한 코드 업데이트에 소요되는 에너지와 시간을 절감

할 수 있도록 하였다.

[0058] 따라서 본 발명에서 설계하고 구현한 EPRCU는 안정적인 시스템을 유지하면서 센서 노드의 부족한 자원을 효율적으로 관리하고 시스템 유지 보수를 위한 효율적인 원격 코드 업데이트 기능을 지원함으로써 무선 센서 네트워크의 다양한 응용 플랫폼을 제공하는 센서 노드용 운영체제로서 적합하다 할 수 있다.

[0059] 지금까지 본 발명을 바람직한 실시예로서 설명하였으나, 본 발명은 이에 한정되지 않고 발명의 요지를 이탈하지 않는 범위 내에서 다양하게 변형하여 실시할 수 있음은 물론이다.

도면의 간단한 설명

[0060] 도 1은 EPRCU의 논리적인 시스템 구성도,

[0061] 도 2는 실행중인 EPRCU 시스템의 메모리 구성도,

[0062] 도 3a는 본 발명에 따른 다중 프로세스 이미지 데이터 시스템에서 다중 스택의 메모리 요구사항을 나타낸 도면,

[0063] 도 3b는 본 발명에 따른 다중 프로세스 이미지 데이터 시스템에서 단일 스택의 메모리 요구사항을 나타낸 도면,

[0064] 도 4는 본 발명에 따른 프로세스 이미지 데이터 제어블록의 구성도,

[0065] 도 5는 본 발명에 따른 커널 api 점프 테이블의 사용 예시도,

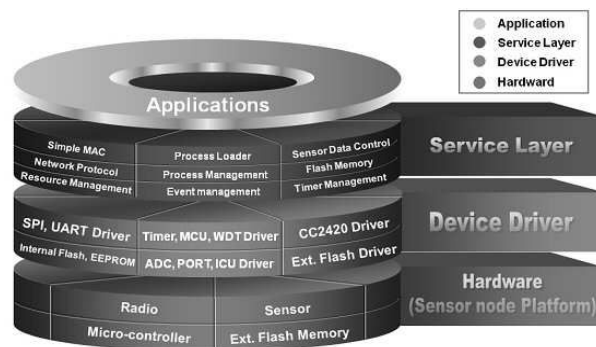
[0066] 도 6은 본 발명에 따른 EPRCU 프로세스 스케줄러의 구조도,

[0067] 도 7은 본 발명에 따른 에이징 타이머를 이용한 프로세싱 에이징을 나타낸 도면,

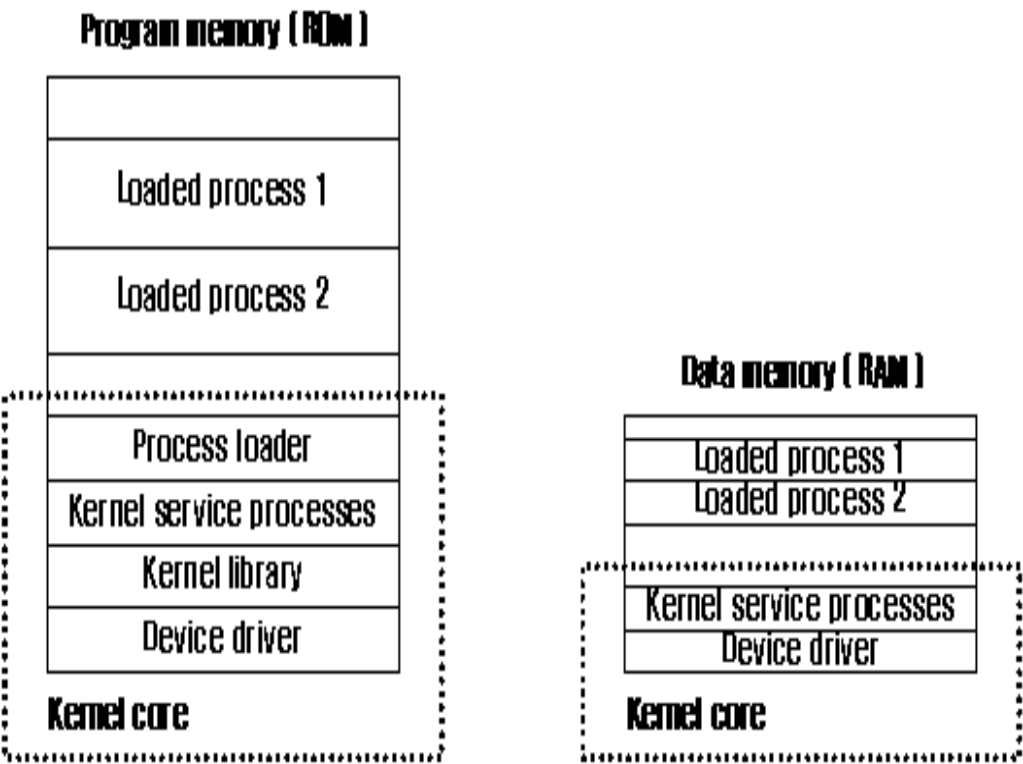
[0068] 도 8은 본 발명에 따른 UTRC SN100 센서 노드와 AVR JTAG ICE mk-II의 구성도이다.

도면

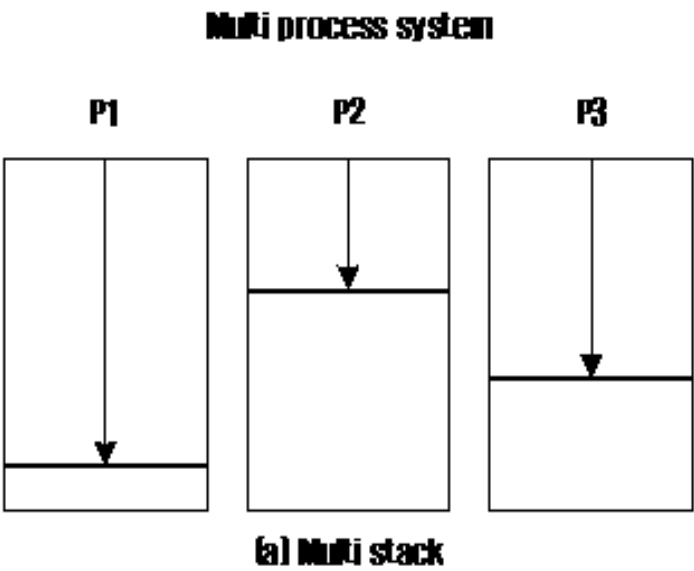
도면1



도면2

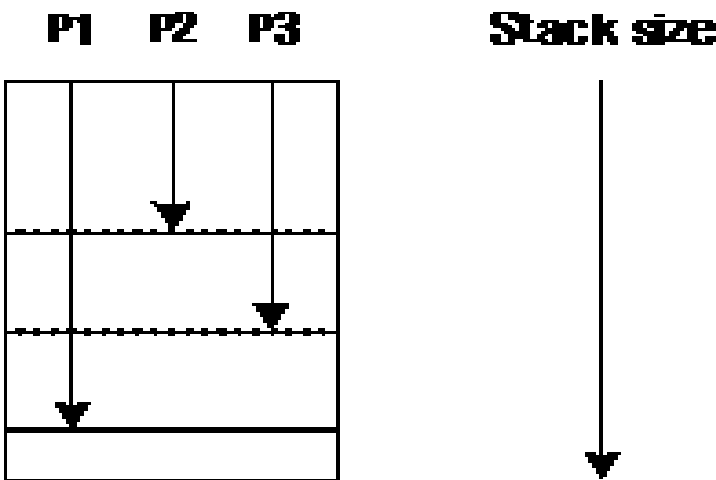


도면3a



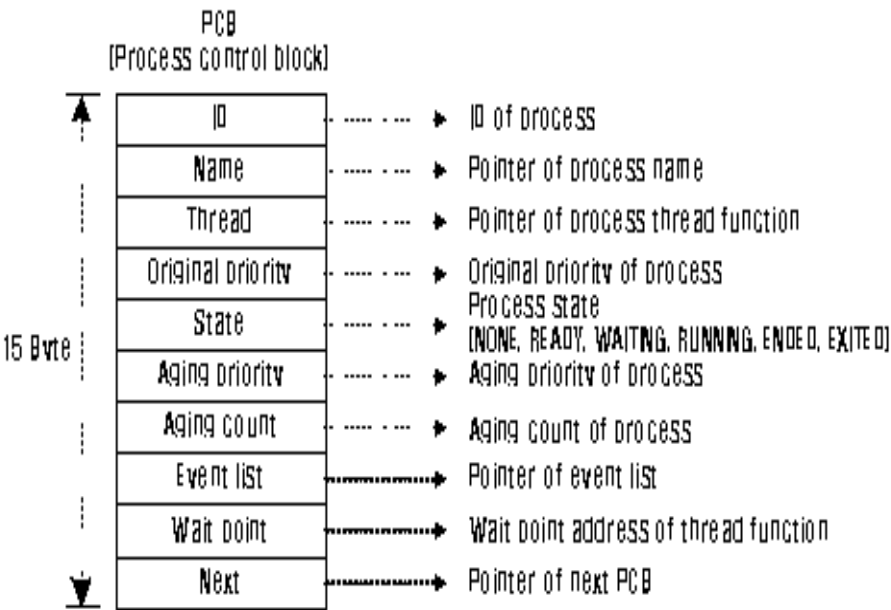
도면3b

Event-driven system

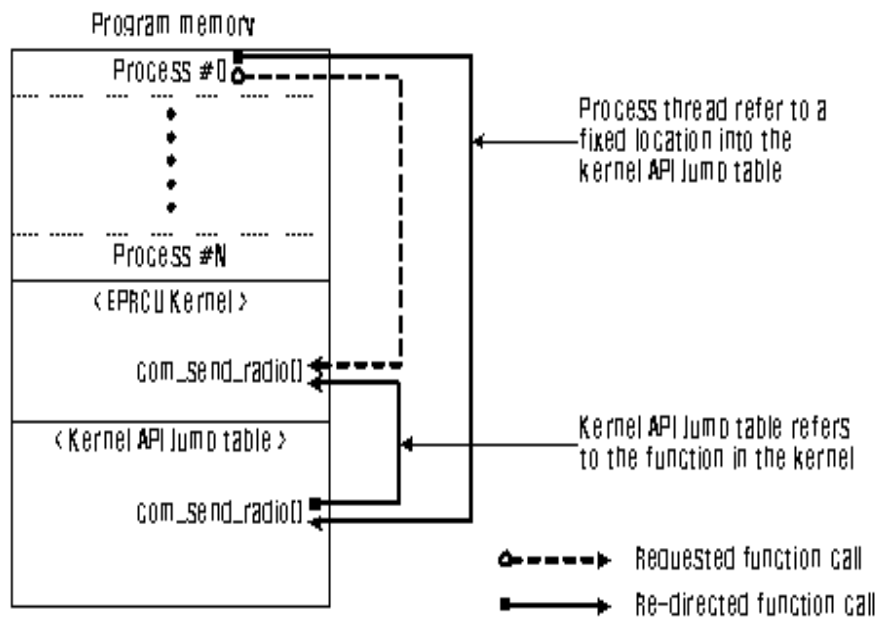


b) Single stack

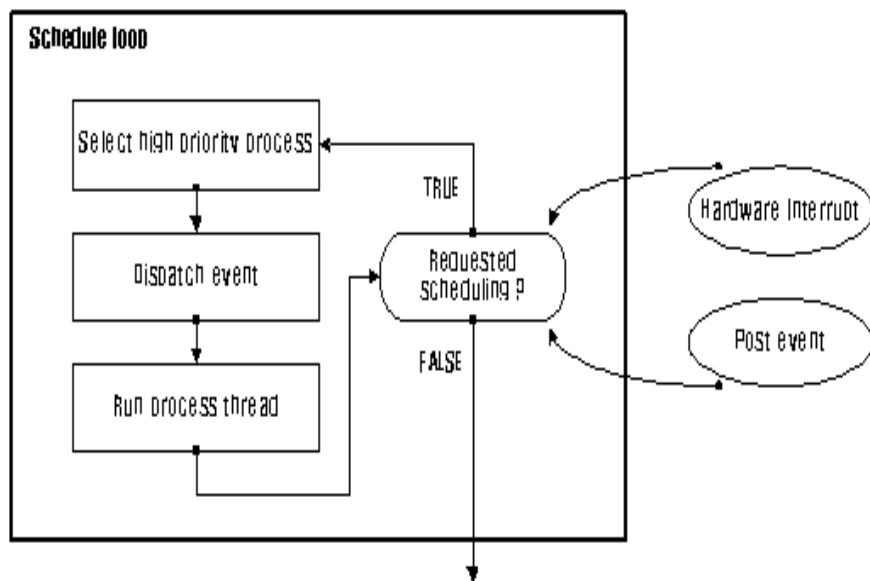
도면4



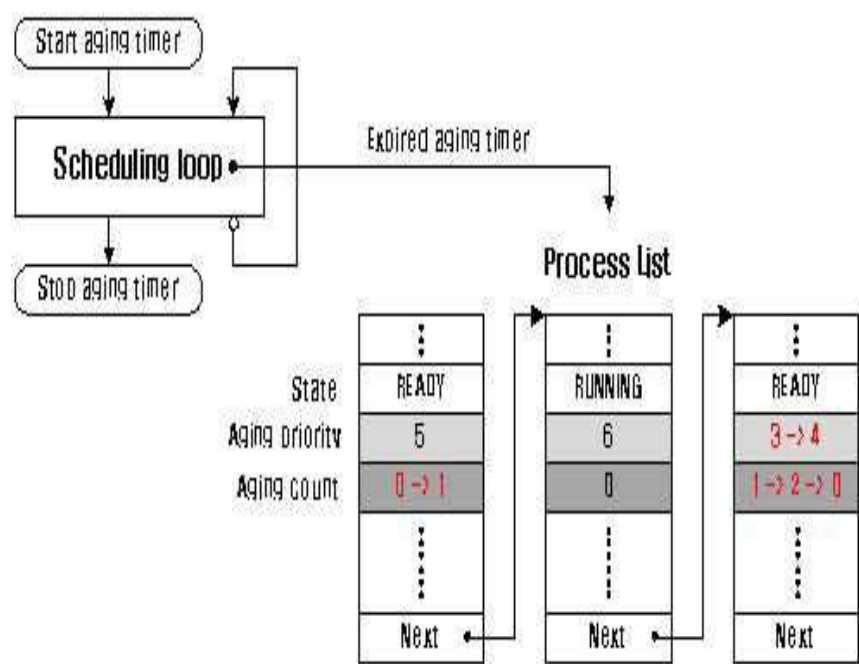
도면5



도면6



도면7



도면8

